X-XSS-Protection

The HTTP **`X-XSS-Protection`** response header is a feature of Internet Explorer, Chrome and Safari that stops pages from loading when they detect reflected cross-site scripting (XSS) attacks. These protections are largely unnecessary in modern browsers when sites implement a strong `Content-Security-Policy` that disables the use of inline JavaScript (`'unsafe-inline'`).

Warning: Even though this feature can protect users of older web browsers that don't yet support CSP, in some cases, **XSS protection can create XSS vulnerabilities** in otherwise safe websites. See the section below for more information.

Note:

- Chrome has removed their XSS Auditor
- Firefox has not, and will not implement X-XSS-Protection
- Edge has retired their XSS filter

This means that if you do not need to support legacy browsers, it is recommended that you use Content-Security-Policy without allowing `unsafe-inline` scripts instead.

Header type	Response header
Forbidden header name	no

Syntax

```
X-XSS-Protection: 0
X-XSS-Protection: 1
X-XSS-Protection: 1; mode=block
X-XSS-Protection: 1; report=<reporting-uri>
```

`0`

Disables XSS filtering.

`1`

Enables XSS filtering (usually default in browsers). If a cross-site scripting attack is detected, the browser will sanitize the page (remove the unsafe parts).

`1; mode=block`

Enables XSS filtering. Rather than sanitizing the page, the browser will prevent rendering of the page if an attack is detected.

`1; report=<reporting-URI>` (Chromium only)

Enables XSS filtering. If a cross-site scripting attack is detected, the browser will sanitize the page and report the violation. This uses the functionality of the CSP `report-uri` directive to send a report.

Vulnerabilities caused by XSS filtering

Consider the following excerpt of HTML code for a webpage:

```
<script>
  var productionMode = true;
</script>
<!-- [...] -->
<script>
  if (!window.productionMode) {
    // Some vulnerable debug code
  }
</script>
```

This code is completely safe if the browser doesn't perform XSS filtering. However, if it does and the search query is `?something=%3Cscript%3Evar%20productionMode%20%3D%20true%3B%3C%2Fscript%3E`, the browser might execute the scripts in the page ignoring `<script>var productionMode = true;</script>` (thinking the server included it in the response because it was in the URI), causing `window.productionMode` to be evaluated to `undefined` and executing the unsafe debug code.

Setting the `X-XSS-Protection` header to either `0` or `1; mode=block` prevents vulnerabilities like the one described above. The former would make the browser run all scripts and the latter would prevent the page from being processed at all (though this approach might be vulnerable to side-channel attacks if the website is embeddable in an `<iframe>`).

Examples

Block pages from loading when they detect reflected XSS attacks:

HTTP:

```
X-XSS-Protection: 1; mode=block
```

PHP:

```
header("X-XSS-Protection: 1; mode=block");
```

Apache (.htaccess):

```
<IfModule mod_headers.c>
  Header set X-XSS-Protection "1; mode=block"
</IfModule>
```

Nginx:

```
add_header "X-XSS-Protection" "1; mode=block";
```

Specifications

Not part of any specifications or drafts.

This lesson is adapted from MDN web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options